

JAVA 的多线程机制



7.1 线程的概念和 **Java** 的多线程机制

7.2 线程的状态与生命周期

7. 线程的优先级及线程的调度

7. 实现多线程的两种方法

7. 线程的常用方法

7. 线程的同步

7. 线程的联合

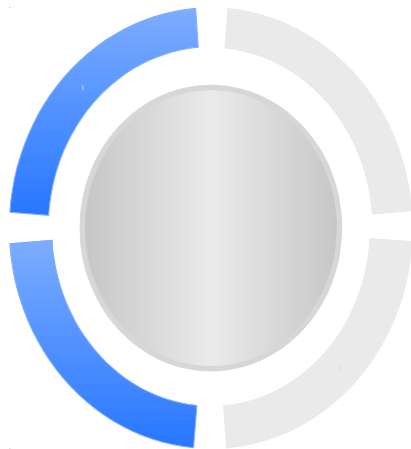


1 线程的概念和 **Java** 的多线程机制



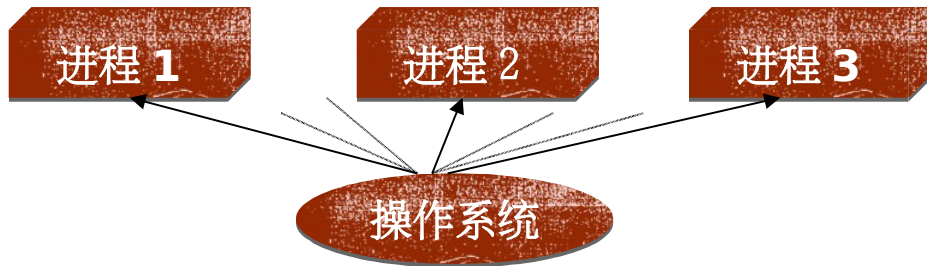
多线程是 **Java** 的特点之一，掌握多线程编程技术，**可以充分利用 CPU 的资源**，更容易解决实际中的问题。

多线程作为一种多任务并发的生活方式，在实际应用中，广泛应用于和**网络有关的程序设计中**，可以**建立清晰的模型**，表现出较大优势。因此，理解 **Java** 的多线程机制，对于 **Java** 面向对象程序设计的学习至关重要。



1.1 操作系统与进程

- 程序是一段静态的代码，而进程则是程序的一次动态执行过程，它对应了从代码加载、执行至执行完毕的一个完整过程，这个过程也是进程本身从产生、发展直至消亡的过程。
- 操作系统可以同时管理计算机系统中的多个进程，让多个进程轮流使用 CPU 资源，甚至可以让多个进程共享操作系统所管理的资源，比如让 Word 进程和其他的文本编辑器进程共享系统的剪贴板。



1.2 进程与线程

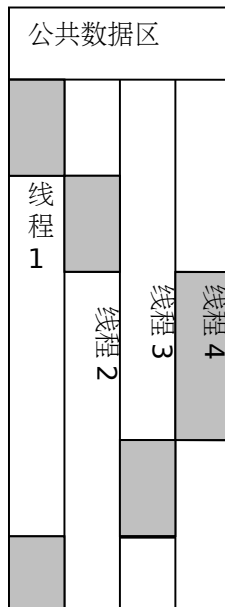
线程是比进程更小的执行单位，一个进程在其执行过程中，可以产生多个线程，形成多条执行线索。每条线索，即每个线程也有它自身的产生、存在和消亡的过程。

1

线程间可以共享进程中的某些内存单元（包括代码与数据），并利用这些共享单元来实现数据交换、实时通信与必要的同步操作。

2

进程



获得 **CPU** 时间



空闲时间



7.1.3 Java 的多线程机制

Java 语言的一大特点就是**内置对多线程的支持**。多线程是指一个应用程序中同时存在几个**执行体**，按几条不同的执行线索共同工作的情况，它使得编程人员可以很方便地开发出能同时处理多个任务的功能强大的应用程序。

1

Java 程序单个顺序的流控制称为线程，**多线程**则指的是在单个程序中可以同时运行多个不同的线程（**多条执行线索**），每个线程执行不同的任务。

2

多线程的程序在运行时，Java 虚拟机快速地把控制从一个线程切换到另一个线程。这些线程将被轮流执行，使得每个线程都有机会使用 **CPU** 资源。

3



1.4 主线程（main 线程）

一

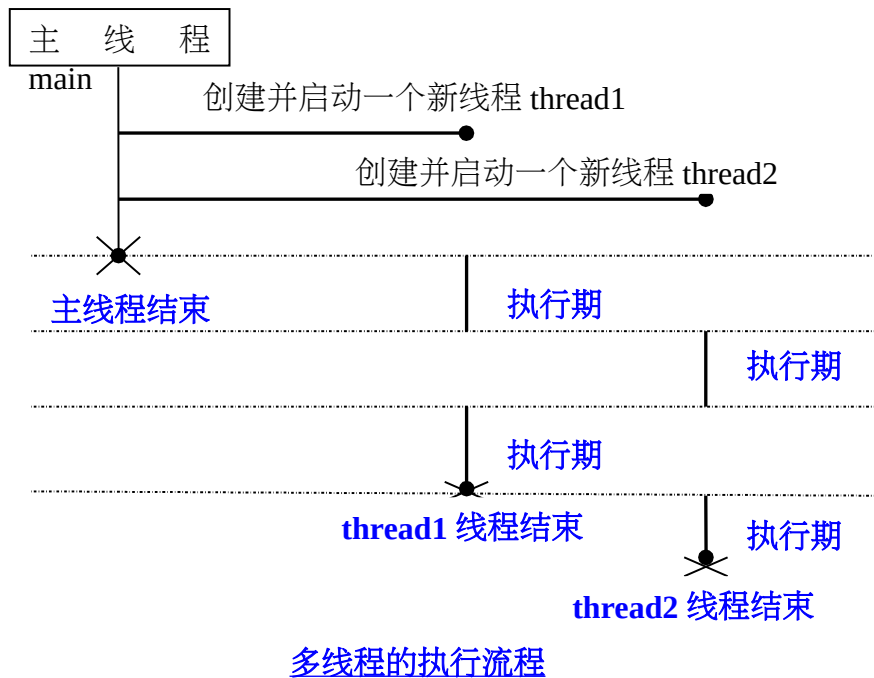
每个 **Java** 应用程序都有一个缺省的主线程。我们已经知道，**Java** 应用程序总是从主类的 **main** 方法开始执行。当 **JVM** 加载代码，发现 **main** 方法后，就会启动一个线程，这个线程称为主线程（**main 线程**），该线程负责执行 **main** 方法。

二

在 **main** 方法的执行中再创建的线程，就称为程序中的子线程。如果 **main** 方法中没有创建子线程，那么当 **main** 方法执行完最后一个语句，即 **main** 方法返回时，**JVM** 就会结束我们的 **Java** 应用程序。

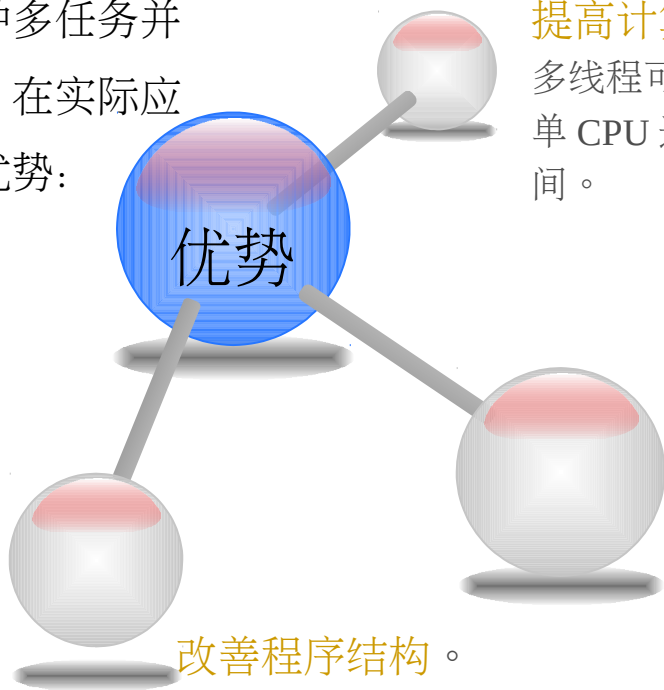


- 如果 **main** 方法中创建了子线程,那么 **JVM** 就要在主线程和子线程之间轮流切换,保证每个线程都有机会使用 **CPU** 资源。
- **main** 方法即使执行完最后的语句,**JVM** 也不会结束 **Java** 应用程序,**JVM** 一直要等到 **Java** 应用程序中的所有线程都结束之后,才结束 **Java** 应用程序。



1.5 多线程的优势

多线程作为一种多任务并发的
工作方式，在实际应用中具有
以下优势：



提高计算机系统 CPU 的利用率。

多线程可以充分利用现代计算机的多 CPU 或单 CPU 运算速度快的特点，从而节省响应时间。

提高应用程序的响应。

尤其是在图形界面程序中，通过多线程技术，将耗时长久的操作放置到一个新线程中执行，而界面仍能正常响应用户的操作，这样可以增强用户体验。

改善程序结构。

一个功能复杂的程序（进程）可以被划分为多个线程，每个线程完成相对独立的功能，使得程序结构清晰、利于理解和修改。



之前我们所接触的程序 (以 Java Application 为例) :

```
public class mainClass
{
    public static void main(String args[])
    {
        第一条语句;
        第二条语句;
        .....
        最后一条语句;
    }
}
```

➤ **main()** 方法是程序执行的入口点, 按语句的顺序, 逐条执行, 执行完最后一条语句, 程序执行完毕。

➤ 以上程序只有一条执行线索, 程序的语句是串行的。程序运行过程中 (进程), 只包含一个主线程, 不存在其它的子线程。



思考：以下程序的第二个循环可以被执行吗？

```
public class mainClass  
{  
    public static void main(String args[])  
    {  
  
    }  
}
```

while(true) { System.out.println(“Hello!”); }

while(true) { System.out.println(“ 大家好 !”); }

以上程序只有一个主线程存在，但：如果建立两个子线程，让每个子线程分别执行一个无限循环，则程序结构清晰，且第二个循环有机会被执行



修改为多线程的程序

```
public class mainClass
{   public static void main(String args[])
    {   // 创建子线程 1, 让其执行第 1 个无限循环 !
        Thread1 th1=new Thread1();
        th1.start();
        // 创建子线程 2, 让其执行第 2 个无限循环 !
        Thread2 th2=new Thread2();
        th2.start();
    }
}
```

以上程序在运行时（进程）包含几个线程呢？

 3 个 : **1** 个主线程, **2** 个子线程



```
class Thread1 extends Thread
{
    public void run()
    {
        while(true)
        {    System.out.println(" Hello!");    }
    }
}
```



```
class Thread2 extends Thread
{
    public void run()
    {
        while(true)
        {   System.out.println(" 大家好!");   }
    }
}
```

